CENTRO NACIONAL DE PESQUISA EM ENERGIAS E MATERIAIS
|
CNPEM
ILUM - ESCOLA DE CIÊNCIA

RELATÓRIO DE APRENDIZADO DE MÁQUINA

TIAMAT — CORRELAÇÃO ENTRE ABALOS SÍSMICOS E O DESVIO DE ORBITA DO FEIXE NO ACELERADOR SIRIUS

ALUNOS:
VINÍCIUS MARIANNO DE MARQUE CUTOLO
FILIPI MARTINS
PROFESSOR:
DANIEL ROBERTO CASSAR

17 DE OUTUBRO DE 2026
CAMPINAS – SP | BRASIL

Resumo

No trabalho a seguir analisa-se dados sísmicos do Sirius e busca tentar criar um modelo capaz de prever quando que uma determinada onda de um terremoto irá causar alguma interferência nos sensores. Para isso, buscou-se criar um algoritmo que prevê chegadas de fases de ondas sísmicas no Sirius e trata o sismógrafo para uma tabela interpretável de dados para, posteriormente, alimentar um modelo de machine learning que busca prever se um terremoto irá influenciar o sismógrafo ou não.

O programa testou oito modelos distintos para essa tarefa, otimizando seus hiper-parâmetros com o optuna e fazendo permutation importance nos valores. O modelo de melhor desempenho foi a regressão logística, com uma precisão de 81.7% e um recall de 33.9%, não atingindo o limiar mínimo de recall $\geq 55\%$. Os resultados apontam para a existência de uma tendencia, mas nenhum modelo superou as metas exigidas, hipotetiza-se que isso seja devido a falta de dados e uma incerteza quanto a precisão dos algoritmos estabelecidos.

Palavras-chave: Algoritmo preditivo, Análise Sísmica, Machine Learning

Introdução

O Sirius é o acelerador de partículas de quarta geração do Brasil, localizado em Campinas. O seu valor cientifico é imenso e, diariamente, fornece imagens extremamente precisas sobre estrutura da matéria e da vida. Para fazer isso, ele utiliza a radiação síncrotron, incidindo-a sobre as amostras. Essa radiação é uma consequência de cargas curvando de forma muito rápido (com velocidades próximas a dal luz), para isso, o Sirius acelera elétrons a mais de 99% da velocidade da luz.

Isso exige trechos retilíneos longos, estabilizadores magnéticos e quadrupolos magnéticas para conseguir manter o feixe concentrado e acelerá-lo de forma controlada. Um desvio desse elétron pode impedir a geração de imagens ou desacelera-lo o suficiente para que seja impossível recuperá-lo, necessitando injetar novos elétrons na linha de luz. Isso é caro e, a depender da oscilação, os corretores de orbita podem diminuir o tempo de vida do feixe tentando alinhá-lo.

Um dos fenômenos que ocasiona esse tipo de evento são abalos sísmicos [1] [2]. O trabalho proposto a seguir buscou analisar se existe alguma forma de usar algoritmos de Machine Learning par analisar e identificar se alguma chegada de um terremoto pode ser mensurado pelos equipamentos do Sirius.

Metodologia

Arcabouço Teórico

Antes de iniciarmos com o algoritmo, precisamos entender como que terremotos ocorrem. O planeta é seccionado em áreas chamadas placas tectônicas, formadas e consumidas por um constante processo de convecção do manto e são relativamente rígidas, compondo a litosfera. Essas placas se movem sobre superfícies mais dúcteis do planeta, a astenosfera [3].

Tal movimentação é uma consequência, principalmente, de resultantes gravitacionais: Ridge Push e Slab Pull [3] [4]. O primeiro ocorre nas regiões dorsais oceânicas, onde a litosfera jovem é muito quente e elevada, favorecendo um afastamento da região. Já o Slab Pull ocorre em placas envelhecidas, frias e densas, que tende a realizar um força que puxa o restante da placa.

Tudo isso contribui para a chamada teoria do rebote elástico [4], o que gera as movimentações subsequentes das placas.

Em geral, as terras superiores a litosfera tendem a seguir esse movimento de forma suave e imperceptível, mas as placas tectônicas compõem blocos distintos, gerando limites onde elas podem interagir. Nos limites existem falhas tectônicas que podem travar durante a movimentação, isso deforma as rochas no entorno, o que acumula tensão na área.

Existem três tipos de limites: limites divergentes, limites destrutivos e limites transformantes. O primeiro ocorre quando duas placas se afastam, o segundo quando elas se movem de encontro e, por fim, o último quando essa movimentação é paralela. Cada uma gera um tipo específico de falha, gerando deformações típicas no ambiente (como as fissuras rift ou dobramentos modernos), mas vale a pena observar um pouco mais as destrutivas, já que, nesse caso, há diferença se são duas placas continentais ou oceânicas, alterando a profundidade e intensidade do terremoto. Em especial, falamos das subducções, que ocorrem no encontro de placas continentais-oceânicas e oceânicas-oceânicas, como a marítima é mais densa, ela facilmente consegue deslizar por debaixo da placa continental, o caso de ambas serem oceânicas isso só ocorre em caso de uma ser muito mais jovem que a outra. Contudo, isso tensiona uma área gigantesca da placa, que, devido à área superficial tensionada ser enorme, é capaz de liberar tremores extremamente fortes. A zona de geração desse tipo de terremoto ocorre nas interfaces das placas, sob uma região chamada de **zona de Wadati-Benioff** [3] [4].

Por isso, o nosso foco será, principalmente nas ondas resultantes de limites destrutivos e subducções. Mesmo assim, isso não significa que terremotos de magnitude elevadas só ocorre nesses regimes, existem outros fatores que podem gerar terremotos de magnitude alta. Uma forma de analisar isso é vendo o momento sísmico (M_0), definido como:

$$M_0 = \mu \times A \times D$$

- μ=módulo de cisalhamento das rochas
- A=área da superfície de ruptura
- D=deslizamento médio

Magnitude é uma terminologia usada para analisar a quantidade de energia liberada durante a ruptura da falha geológica. O momento sísmico, definido anteriormente, é uma das formas de colocar essa grandeza em escala (denominada de $M_w = \frac{2}{3}(log_{10}M_0 - 9.1)$) [5]. Dentro desse trabalho, essa definição será particularmente importante para terremotos de magnitude 7 para cima, entretanto, exitem outras escalar que irão aparecer nos dados. Para elas, siga para a referência [5].

Vale salientar que existe uma espécie de quarta categoria para terremotos, que são as intraplacas, que ocorrem em regiões frágeis ou com falhas antigas das placas. Vale lembrar que, mesmo que muito raro no restante do mundo, esse é o tipo de terremoto mais predominante no Brasil.

No fim, isso pode parecer irrelevante àqueles que já espiaram o restante do relatório. Porém, conhecer sobre falhas permite, ao analisarmos as ondas P e S, tirar conclusões sobre o mecanismo focal do terremoto se soubermos quais fases tem um impacto significativo no Sirius. Além disso, precisamos lembrar que o modelo usado aqui é uma simplificação, os fatos mencionados são parâmetros que nao afetam diretamente a previsão, com a geometria, seria possível, inclusive, prever a intensidade de chegada de ondas internas.

Em geral, um terremoto gera quatro tipo de ondas que vão viajar o planeta: ondas P, ondas S, ondas Rayleigh e ondas Love, com as duas primeiras sendo internas e as duas últimas superficiais. Cada qual será desenvolvida posteriormente.

Ondas internas

Ondas internas são aquelas que usam de algumas propriedades mecânicas do material. As ondas P são ondas de compressão, elas vem de segmentos de material se comprimindo e afastando outros, propagando uma onda de pressão pelo planeta. Ondas P são mais rápidas do que ondas S, veja a Figura 1.

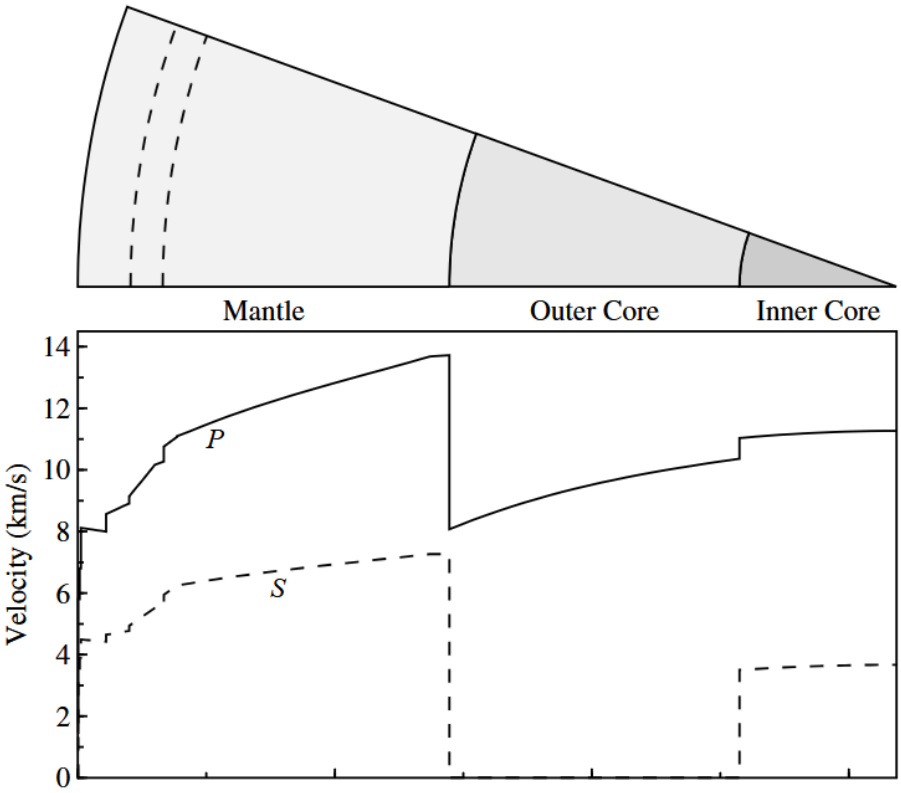


Figura 1: Velocidades das ondas P e S e densidade da Terra em função da profundidade. Os valores são apresentados com base no Modelo de Referência Preliminar da Terra (PREM, na sigla em inglês), de Dziewonski e Anderson (1981); exceto por algumas diferenças no manto superior, todos os modelos modernos da Terra aproximam-se desses valores.

Fonte: Figura 1.1 encontrada em [3].

As ondas S são resultantes das componentes de cisalhamento do terremoto. Ambas podem parecer muito iguais se analisarmos somente o seus trajetos, vide Figura 2. Contudo, existem pequenas diferenças na propagação, ângulo de saída e potência transmitida e uma grande diferença entre elas: a falta de ondas S no núcle externo. Pontos que serão discutidos mais posteriormente nesse relatório.

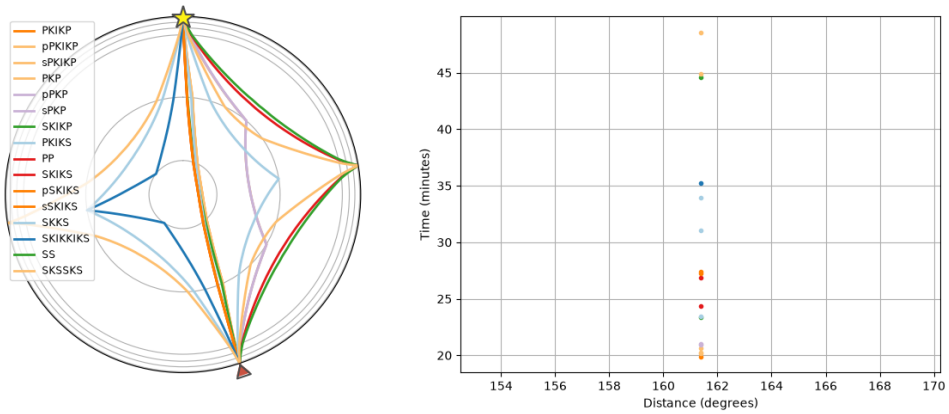


Figura 2: Plot das diferentes fases internas. A esquerda, temos o trajeto das fases, separadas por coloração e caminha, nessa imagem temos a estrela representado a origem do evento e a seta representando o alvo. Já a direita temos as chegadas organizadas por ordem temporal, mantendo o esquema de cores da imagem a esquerda

Fonte: elaborado pelo grupo.

Nesse trabalho, foi usado um modelo simplificado do planeta para analisar o deslocamento de cada uma das fases das ondas, considerando a terra como um sistema unidimensional no quesito densidade (dentro de um plano polar). Com isso, basta analisar o desvio de ondas ao caminhar por uma direção de índice de refração variável, além dos próprios desvio ao transitar de região.

A depender do caminho, uma onda pode obter uma nomenclatura distinta, por isso, se olharmos a Figura 2, notamos isso. Além das ondas P e S, temos:

- K: um trecho de P no núcleo externo
- I: um trecho de P no núcleo interno
- J: um trecho de S no núcleo interno
- c: reflexão no limite manto — núcleo externo
- i: reflexão no limite núcleo externo — núcleo interno
- p: onda P subindo da fonte até a superfície
- s: onda S subindo da fonte até a superfície

Não temos uma letra para ondas S noo núcleo externo pois ele é líquido, logo, esse tipo de onda não se propaga lá.

Ondas superficiais

As ondas superficiais são divididas em duas categorias: Rayleigh e Love. As ondas do primeiro tipo ocasionam um movimento elíptico da superfície, já as do segundo se movem perpendicular a direção de propagação e na horizontal, como visto na Figura 3. Mesmo que ambas pareçam semelhantes, existem exigências distintas do seu meio de propagação, Rayleigh são ondas mais permissivas, caminhando em superfícies livres e elásticas, já as Love exigem um

meio estratificado para continuarem existindo. Isso também restringe a energia dessas ondas, mudando a velocidade de grupo.

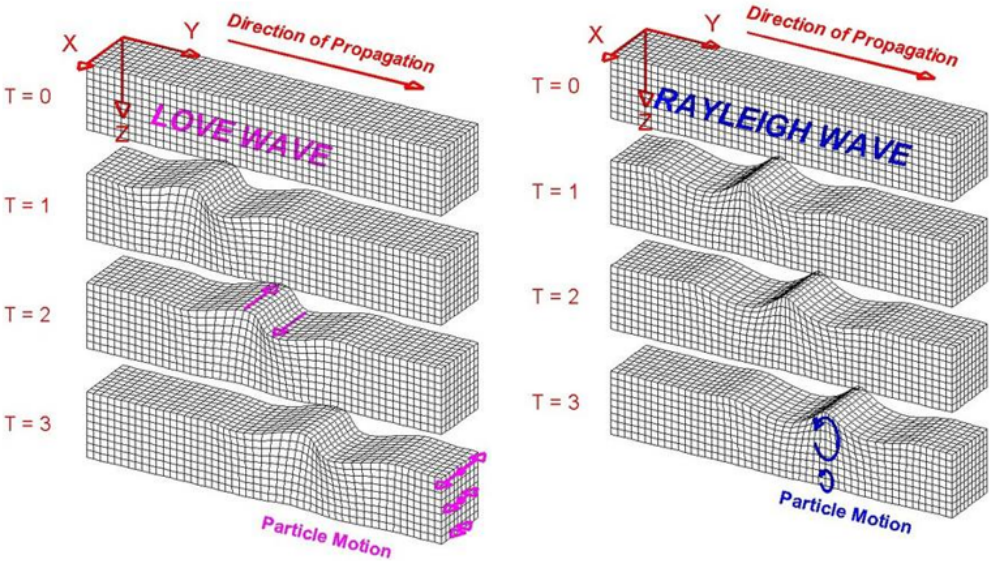


Figura 3: Padrão de movimento das ondas Rayleigh e Love
Fonte: disponível em [6].

Além disso, essas ondas são geradas pelas diversas oscilações do terremoto, que atira uma espectro largo de frequências que é posteriormente dispersado pelo meio, lançando grupos de diferentes frequências e, por consequência, com diferentes velocidades de grupos. Como estamos tratando de um modelo simplificado do planeta, não existe preferencia de caminho da onda e nem possíveis refrações ou reflexões na transição da terra ao oceano.

Isso significa que o tempo de chegada, nesse caso, é majoritariamente definido pela velocidade de grupo e pela distância do epicentro. Por isso, os tempos de chegada das ondas foi calculado usando:

$$t_{onda} = \frac{L}{U_{onda}(T)}$$

onde:

- t_{onda}: é o tempo de percurso daquela onda
- L: é a distância percorrida pela superfícies
- U_{onda}: velocidade de grupo
- T: período da onda

Aquisição de Dados

Existem dois locais principais de aquisição de dados pelo grupo: catálogo de terremotos da USP e o banco de dados do Sirius.

A ferramenta usada, o `ObsPy` tem uma função que permite buscar dados de terremotos em catálogo específicos [7]. Por uma questão de confiabilidade e de recomendação dos operadores da sala de controle, foi utilizado o catalogo da USP como fonte de informações sísmicas.

Dessa fonte, foram recolhidas as seguintes informações:

- nome
- latitude
- longitude
- profundidade
- magnitude
- tipo de magnitude
- tempo de origem do evento

Já para os sismógrafos e os Beam Position Monitors (BPMs) foram obtidos diretamente do LNLS Archiver. Usou-se a biblioteca `requests` para baixar todos os registros que continham a palavra **terremoto** ou **sísmico**. Como esses eram arquivos de texto, seria, teoricamente, necessário ler todos e anotar os tempos de início e de fim do evento, que estavam no texto. Para poupar esse trabalho, integrou-se um Large Language Model (LLM), especificamente o **gemini 3.5 flash lite** por uma chave de API, para ler os logs e retornar os tempos de início e fim dos eventos.

Em seguida, o arquivo baixa tabelas de dados dos três eixos dos sismógrafos e dos dois eixos dos BPMs [8]. Isso é armazenado em uma pasta, que servo como um banco de dados para analises posteriores.

Vale de ressaltar que os amostradores dos sismógrafos e dos BPMs operam com período de amostragem de 0.01 segundos [9]. Porém, a variação temporal dos dados usados é de apenas 1 segundo, isso é uma consequência do optimizador nativo do Archiver, que permite escolher o número de pontos que uma amostra irá retornar [8]. Esse número foi escolhido propositalmente para retornar intervalos de 1 segundo, pois os dados observados eram muito volumosos e apresentavam ruído expressivo.

É importante mencionar que os dados só foram pegos de registros que visualizaram uma assinatura de um terremoto. Logo, existe um viés de amostra dentro dessa análise, por isso, mesmo que hajam chegadas que aparentem ter pouco ou nenhum impacto nas oscilações, o número de exemplos negativos é muito baixo perante ao número de exemplos positivos.

Tratamento de Dados

Terremotos

Dentro da perspectiva da terra unidimensional, é possível estimar a chegada de todas as ondas por meio do cálculo diferencial-integral. Para isso, é necessário separar como ondas de corpo se diferenciam de ondas de superfície.

Cada um dos modelos faz uma consideração diferente da terra, o módulo de ondas superficiais usado [10] considera uma terra perfeitamente esférica com **spherical splines** (curvas suaves em um espaço esférico que acompanham a superfície) para modelar as lentidões ocasionadas por condições geográficas. Já o módulo de ondas de corpos ataca o problema com cálculo diferencial integral e um modelo unidimensional da terra, analisando como que ondas se curvam em um meio de índice de refração variável e fronteiras não suaves desse índice [11]. Ambos desprezam dissipação de energia, logo, a velocidade da onda é somente dependente do meio.

A formula geral usada nesse modelo é:

$$t = t_0 + \frac{D}{U(\rho, \omega)}$$

Em que:

- t = o tempo de chegada em algum sistema temporal específico (no caso desse projeto, no UTC-0).
- t_0 = tempo de origem do evento.
- D = distância, em quilômetros, entre a origem do evento e o alvo das ondas. Essa distância deve estar contida na geometria terrestre, então, sua definição seria a geodésica entre a origem e o objetivo, com as superficiais sendo necessariamente ondas contidas em um espaço esférico e as ondas de corpo em um espaço sem curvatura(somente sua fronteira apresenta tal propriedade).
- $U(\rho, \omega)$ = velocidade de grupo do pacote de ondas emitido.

O primeiro modelo calcula as diversas chegadas das ondas Rayleigh e Love, retornando a frequência de cada uma por considerar a geometria da emissão de onda. O segundo não tem a mesma capacidade de retornar as frequências da chegada, já que desconsidera a geometria do evento de origem e usa uma representação esférica com densidade tendo simetria radial da terra. Isso resultará em uma suposição que terá de ser feita posteriormente.

Por conta dessa simplificação, existe uma incerteza de previsão de chegada do segundo modelo. Segundo a sua documentação [10], o modelo tem uma incerteza de ±5 segundos da chegadas das ondas de corpo. Isso pode aparentar ser pouco, mas estudos sobre o formato das ondas P e S demonstram que elas tem uma incerteza de chegada de 0.1 segundo e um período que pode variar entre 20 e 5 segundos (0.05 Hz e 0.2 Hz) [12]. Isso significa que ao analisar uma chegada e um pico no sismógrafo, não existe forma de saber como que aquela onda interferiu ou se ela que ocasionou a oscilação.

O problema exposto é um resíduo persistente na análise, para corrigi-lo seria necessário incrementar o modelo e seus cálculos, utilizando gradientes com filtro de ruído para simular pequenas imperfeições na terra, além de ponderar regiões moveis ou de densidade regional.

Para permitir com que o modelo tenha uma consistência maior na previsão de impacto de ondas, usou-se um intervalo de 10.2 segundo de tolerância a partir de uma chegada. Isso permite, teoricamente, que ondas realmente influentes possam conter o pico de resposta, mas também permite que outras chegadas próximas sejam confundidas como ondas influentes.

Sismógrafos

Os dados retornados do sismógrafo tinham um formato semelhante ao visto na Figura 4.

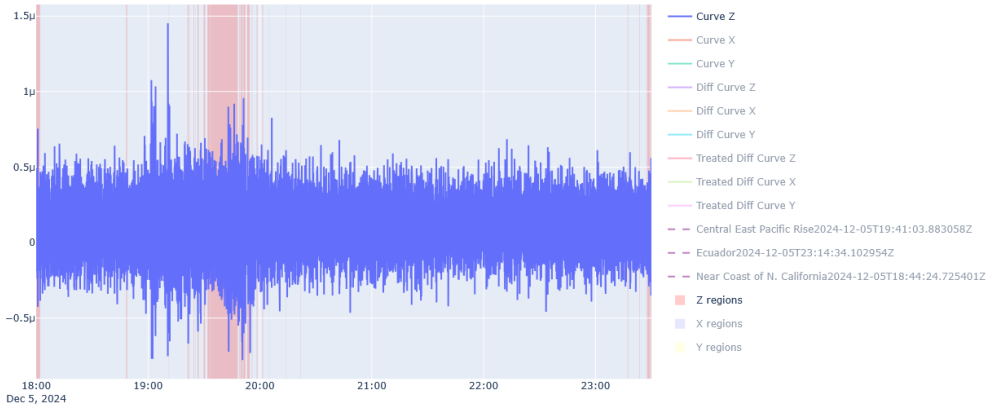


Figura 4: Dados do eixo Z do sismógrafo do Sirius no dia 5 de Dezembro de 2024, entre 18:00 e 23:59.

Fonte: plot de autoria própria com os dados disponíveis em LNLS Archiver.

Isso dificulta uma análise por um modelo de aprendizado de máquina, já que o sismógrafo não só oscila, ele estabiliza em regiões fora do zero quando um pico de uma onda está atravessando o local. Por isso, olhar para os dados puros impediria uma análise detalhada de em que pontos isso demonstra uma chegada real de uma onda e que ponto existe uma excitação do sensor e em que ponto existe um relaxamento real ou uma chegada em quadratura.

Como resolução, um algoritmo de tratamento e filtro por frequência foi proposto, o seu objetivo é transformar oscilações no sismógrafo em uma variável gráfica e filtrar a frequência da amostra, já que tipos de ondas diferentes ocorrem em escalar temporais distintas.

Vale desenvolver uma função denominada de `normalized_envelope`, que realiza a operação proposta acima. Ela analisa a frequência de amostragem e remove o offset dos dados originais como uma forma de pré-tratamento. Em sequência, entra o filtro passa-banda (a função `butter`), que preserva oscilações dentro das frequências exigidas pelas ondas (representadas pelos argumentos f_{min} e f_{max}). Na prática, isso é analisado com o auxílio da composição espectral da série de Fourier [13]. Em sequência, um filtro causal é aplicado para que o `butter` corte frequências indesejadas, contudo, isso geraria um atraso de fase, o que é inaceitável no contexto do projeto. Por isso, aplica-se o `sosfiltfilt`, que aplica uma filtragem para frente e para trás, cancelando o deslocamento.

A transformação de Hilbert ajuda a descobrir as amplitudes reais das oscilações, mesmo que a tal amplitude seja uma função dependente do tempo. Por fim, usa-se esse resultado para calcular o ruído de cada ponto para removê-lo da onda resultante. Assim, resultados como os vistos na Figura 5.

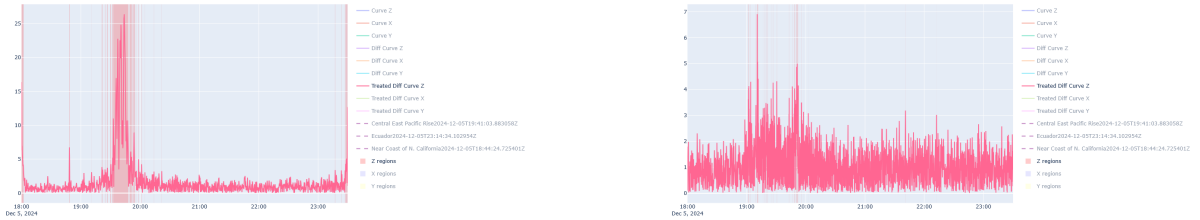


Figura 5: Representações gráficas do processo descrito anteriormente. A esquerda, o gráfico resultante de um tratamento usando as frequências superficiais (que tendem a ser bem menores que as de corpo) e a direita usando as frequências observadas em ondas de corpo

Fonte: plot de autoria própria.

Sistema de correlação de terremotos

Agora, como definir se uma chegada de um terremoto corresponde a uma amplitude observada ou se não houve interferência alguma? Primeiro, deve-se expandir o intervalo em uma hora para frente e para trás do período analisado do sismógrafo. Isso permite buscar no catalogo diversos terremotos dentro desse horário para que seja obtido todas as chegadas possíveis. Em seguida, filtra-se especificamente as chegadas que estão dentro do intervalos disponível para análise. O resultado do processo descrito pode ser visto na Figura 6.

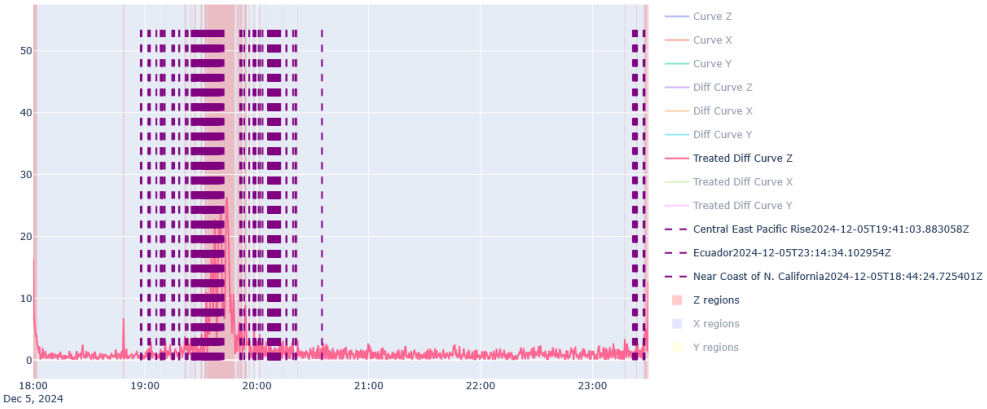


Figura 6: Imagem exposta na Figura 5, especificamente a da esquerda, sobreposta com as diversas chegadas dos terremotos ocorridos no equador, na costa norte da califórnia e no centro-leste do Oceano Pacífico na mesma data do registro da Figura 4.

Fonte: plot de autoria própria.

Agora, torna-se necessário definir algum sistema para classificar cada chegada de terremotos. Para isso, usou-se três valores como métricas: um sistema de score, um p-valor e um effect-z.

Para calcular isso, existem duas funções relevantes no script `association_v2`: `_union_mean` e `calculate_associations_random`. A primeira calcula a atividade média dentro de um intervalo de tolerância, como prevenção para repetições; a função marca todos os intervalos e, em seguida, cria uma máscara contendo todos os valores que pertencem a algum desses intervalos.

Em seguida, o código cria deslocamentos aleatórios das amostras e verifica se eles são válidos, com válidos significando: não sobrepõem um intervalo existente e não saem do intervalo de medidas. Isso permite calcular a atividade média dentro de regiões que, teoricamente, não têm nenhuma chegada prevista. Obviamente, isso parte de um pressuposto de que o modelo proposto para avaliar as ondas de terremotos consegue encontrar todas as chegadas; assim, fora dos intervalos originais, a atividade sísmica seria apenas relaxamento do sismógrafo ou ruído. Esse processo é auxiliado pela função `event_blocks`, que define regiões proibidas associadas a terremotos reais.

Por fim, a função `calculate_associations_random` realiza todo o processo descrito anteriormente e calcula as estatísticas. Para isso, os argumentos passados para as funções citadas até agora são uma junção de objetos filtrados por nome de evento e fase, isso significa que se existir mais de uma chegada de uma fase ela sera agrupada no mesmo objeto de análise. Isso foi uma decisão proposital, como fases iguais de um terremoto tendem a chegar muito próximas, o algoritmo feito até o momento não é capaz de separa-las. Por isso, elas serão tratadas como um 'caso especial' (nessa situação, significaria que os seus intervalos se sobrepões, funcionando como uma única chegada com tolerância maior).

Assim, é possível calcular as métricas mencionandas. O score é uma medida de quantos controles tem atividade média menor do que o alvo, ou seja, score $= P(C < \text{obs})$. O p-valor calculado é, na realidade, um p-valor empírico de cauda superior, ele verifica com que frequência uma janela aleatória produz uma atividade maior ou igual que a observada, calculado como $p_{\text{valor}} = \frac{1+\sum(C \geq \text{obs})}{1+|C|}$. Por fim, o effect-z é uma métrica de diz qual o tamanho do desvio daquela medida em relação ao controle, calculado como effect-z $= \frac{\text{obs}-\bar{C}}{\sigma_C}$.

Vale ressaltar que as duas primeira medidas mensuram frequência e serão particularmente úteis em modelos de classificação, já o effect-z é uma medida de amplitude e poderá ser utilizada para modelos de regressão continua.

Existe uma última função dentro desse script que é a `null_calibration`. Essa função não opera em conjunto com as outras, mas ela valida a qualidade das medidas dadas anteriormente. Ela cria um certo número de eventos falsos (deslocando eventos reais para regiões sem atividade) e calcula o quantas vezes o modelo encontra novas associações. Se mesmo para esses deslocamentos o p-valor, por exemplo, continuar baixo, isso pode significar que o modelo está produzindo muitos falsos positivos. Contudo, não se pode confundir essa função como um 'validador de terremotos', ela verifica se o método produz falsos positivos conscientemente, não comprova correlação entre os eventos e as chegadas.

Posteriormente, tudo isso é unido em uma única tabela, que contém informações do terremoto necessárias para treinar um modelo e os targets. Três informações compostas foram criadas com o objetivo de dinamizar a expressão dos dados. Por isso, a coluna `log_depth` é definida como $ln(1 + \text{depth})$, a coluna `log_dist` é definida como $log_{10}(\Delta R_{\text{Terra}})$ e `expected_amp` como $M - log_{10}(\Delta)$.

A tabela resultante seguirá a ordem:

Tabela 1: Ordem dos elementos da tabela de modelo resultante. Note que existem, no total, 10 atributos de base e 3 derivados, três identificador (event_id, first_arrival e last_arrival) e quatro targets.

event_id	phase	depth	incident_angle	takeoff_angle	...
log_depth	axis	axis_noise	activity_order	magnitude	...
expected_amp	magnitude_type	epicenter_distance	log_distance	first_arrival	...
last_arrival	ray_param	score	p_value	effect_z	...

Modelos De Aprendizado

Foram selecionados oito modelos bastante suportados e usados na literatura de machine learning: Regressão Logística (LogReg), Árvore de Decisão, Floresta Aleatória, HistGradient Boosting e Support Vector Classifier (SVC), XGBoost, K-Nearest-Neighbor (KNN) e Navier Bayes, além do baseline criado usando Dummie Classifier. Cada modelo buscou representar diferentes famílias de classificadores ou simplesmente testar modelos vistos durante a disciplina de aprendizado de máquina da ILUM - Escola de Ciências 2026, ministrada pelo Dr. Daniel Roberto Cassar.

Regressão Logística

Esse modelo teve dois principais motivos para ser selecionado. Ele se apresenta como um bom candidato para ser um modelo baseline de classificação, sendo mais complexo que um modelo de regressão linear sem perder o seu caráter interpretável. Ademais, ele foi usado em um outro trabalho pelo autor, existindo uma certa preferencia pessoal dele [14] [15].

Árvore de Decisão

Esse é mais uma representação didática. Não existe uma esperança muito grande de que ele vá conseguir ser melhor que a Floresta Aleatória. Contudo, ele é um dos modelos didáticos aprendidos em sala de aula, servindo como uma abordagem muito mais focada no lúdico dos métodos vistos em aula. Contudo, existe uma possibilidade dela ter algum sucesso, já que arvores de decisão tem uma boa capacidade de representar dados não lineares.

Floresta Aleatória

Aqui começamos com candidatos mais fortes a resolução. Como dito anteriormente, arvores de decisão são bons candidatos para avaliar dados não-lineares e interação entre as variáveis. Além disso, florestas são muito mais resistentes a overfitting, resultando em um potencial candidato para a familia de algoritmos de bagging.

HistGradient Boosting

Esse algoritmo representa a família de métodos de boosting, junto com o XGBoost. Enquanto florestas aleatórias criam árvores de forma independente, o Gradient Boosting cria sequenciamento árvores visando diminuir o erro, ele permite avaliar se o problema se beneficia de modelos aditivos [14] [16].

SVC

O SVC foi selecionado pela sua capacidade de maximizar margens entre classes. Ele apresenta uma perspectiva diferente dos modelos baseados em árvore e tem um potencial muito alto por conseguir construir fronteiras altamente não-lineares

XGBoost

Esse modelo vem como uma espécie de sucessor, já que ele apresenta um histórico mais consolidado dentro da literatura para problemas de classificação com dados tabulares. Ele funciona de modo similar ao HistGradientBoosting no quesito de criação de árvores, mas ele visa

minimizar o erro acumulado de árvores anteriores. Sua inclusão é especialmente interessante na comparação com o seu semelhante já citado, além de ser uma ferramenta que lida melhor com overfitting [14] [17].

KNN

Mais um modelo que visa testar os conhecimentos de aula. Não existe uma particular esperança para ele, já que acredita-se que o nível de complexidade dos atributos não possa ser reduzido a distâncias no espaço. Contudo, ele serve, também, com um representante de modelos baseados em distância, podendo servir como analise dessa família.

Navier Bayes

Esse modelo é um caso particular. Será usado o a variação Gaussiana dele, que tem uma capacidade mais interessante de previsão em modelos de classificação. Contudo, ele é feito para atuar dentro de cenários onde os dados se distribuem de forma gaussiana, o que, possivelmente, não é o caso do problema tratado. Contudo, ele é um representante da abordagem probabilística do problema, sendo, possivelmente, o melhor dentro dela para ser abordado aqui [14] [18].

Treinamento

O modelo é definido em um pipeline de duas etapas, o primeiro é um `ColumnTransformer` que aplica `StandardScaler` nas variáveis numéricas e `OneHotEncoder` nas categóricas. Variáveis que não fossem explicitadas nesse modelo eram excluídas, facilitando o teste de ablação. Em seguida, o modelo em sí era aplicado.

Como target, foi usado uma classificação binária entre terremotos 'influentes' (aqui, o termo terremoto se refere a combinação entre evento, fase e eixo) e 'não influentes'. Essa é uma classificação artificial criada pelo autor como forma de tentar entender quando que uma oscilação aparenta ser um resultado de alguma oscilação e quando não aparenta. Para separar os grupos, usou-se o p-valor como métrica: terremotos com p-valor menor que 10% são taxados como anômalos e, portanto, 'influentes'. Para treinar o modelo, os dados foram separados em grupos estratificados de cada abalo. Com isso, não houve Data Leakage entre fases correlacionadas do mesmo terremoto.

Para realizar a otimização de hiper-parâmetros, foi definido o script `optuna_searches`. Como muitos testes estavam sendo propostos simultaneamente, fez-se a validação usando o `GridSampler` do optuna para fazer uma busca em grade, validando para estimar o desempenho do modelo variando não só os hiper-parâmetros mas também o limiar de ativação e definir filtros específicos para classificar candidatos (esses serão desenvolvidos numa próxima seção).

Os intervalos de busca foram escolhidos como uma forma de explorar o modelo em um intervalo amplo de possibilidades, buscando avaliar, em geral, em qual regime o modelo se comporta melhor. Segue uma lista sobre a motivação individual de cada lista:

- Regressão Logística:
  - C: buscou-se avaliar o modelo em um regime de regularização forte (valores mais baixos) e regularização fraca .
  - l1_ratio: como está sendo usado a penalização por ElasticNet, testou-se combinações igualmente espaçadas entre l1 e l2 para descobrir se o modelo preferia uma regularização ou outra.

- Árvore de Decisão:
  - max_depth: buscou-se limitar a profundidade do modelo para árvores mais rasas (para caso houvesse overfitting) e outras mais profundas (caso os dados permitissem).
  - min_samples_leaf: testou-se valores menores para aumentar a sensibilidade e maiores para caso houve-se problema com overfitting ou ruído.
  - ccp_alpha: valores crescentes foram testados para verificar se retirar ramos pouco relevantes ajudam na capacidade de generalização da árvore.
- Floresta Aleatória:
  - max_depth: buscou-se limitar a profundidade do modelo para árvores mais rasas (para caso houvesse overfitting) e outras mais profundas (caso os dados permitissem).
  - min_samples_leaf: testou-se valores menores para aumentar a sensibilidade e maiores para caso houve-se problema com overfitting ou ruído.
  - max_features: foram avaliadas tanto a configuração clássica (sqrt) quanto diferentes frações do conjunto de atributos, permitindo analisar o impacto da aleatorização na redução da correlação entre árvores.
- HistGradientBoosting:
  - learning_rate: Valores menores promovem aprendizado mais gradual e frequentemente exigem maior número de iterações.
  - max_iter: A combinação entre learning_rate e max_iter permite avaliar diferentes níveis de complexidade efetiva do modelo.
  - max_depth: foi limitado a valores baixos porque algoritmos de boosting geralmente apresentam melhor desempenho com árvores rasas.
  - l2_regularization:controla a magnitude das atualizações do modelo, reduzindo a probabilidade de overfitting.
- Support Vector Classifier:
  - C: foram testados valores menores para ter um ajuste mais suave e regularizado até valores maiores que ter fronteiras mais complexas
  - gamma: um espaço foi testado para averiguar superfícies mais globais e outras mais localizadas
- XGBoost:
  - learning_rate: Valores menores promovem aprendizado mais gradual e frequentemente exigem maior número de iterações.
  - n_estimator: número total de árvores construídas, testando valores menores até os maiores para evitar overfitting.
  - max_depth: foi limitado a valores baixos porque algoritmos de boosting geralmente apresentam melhor desempenho com árvores rasas.
  - subsample: introduz aleatoriedade nas amostras ao utilizar subconjuntos dela, ajuda a reduzir a variância do modelo.

- K-Nearest Neighbors:
  - n_neighbors:foi testado uma faixa ampla para averiguar desde classificadores locais até os globais.
  - weights: foi testado ponderações uniformes e baseadas em distância para entender como os dados se estruturam.
  - metric: foram testadas diversas métricas, já que o problema é multidimensional e não se sabe se os dados são afetados por isso.
- Gaussian Naive Bayes:
  - var_smoothing: foi testado variações logarítmicas para averiguar situações com regularização desprezível e outras com regularização forte.

Resultados e discussão

Após a aquisição e tratamento de todos os terremotos obtidos, reuniu-se 49 terremotos distintos, com 3429 chegadas registradas (ao leitor atento, nota-se que ao somar os valores das matrizes de confusão obtêm-se 2931 registros, isso é uma consequência de tr es momentos tendo tido erro na atribuição de controles, resultando em entradas do tipi NaN). Usando o split proposto, isso fornece 1527 chegadas taxadas como 'influentes' e 1404 como 'não influentes'. Vale ressaltar que isso não significa que metade das ondas que chegaram não tem importância alguma nas oscilações vistas, dificilmente isso poderia ser afirmado de um resultado que tem um p-valor=11%.

Para visualizar melhor esse paradigma, olhe para a Figura 7. Parece que existe alguma preferência por quais fases geram resposta dentro do Sirius, fases como PKKP sempre estão perto de locais em que grande oscilações foram observadas, mas ondas P aparentam ser bem menos influentes.

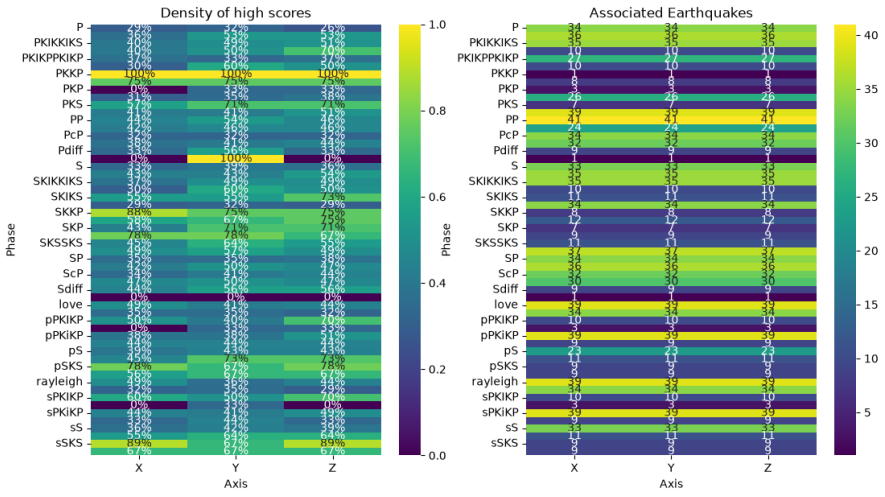


Figura 7: Mapa de calor que compara frequência de classificação como uma onda influente, observado na tabela da esquerda, e o número de terremotos em que aquelas fases aparecem.

Fonte: plot de autoria própria.

Além disso, é possível buscar alguma correlação clara entre os atributos que são fortes candidatos para interferência no score dado. Para isso, observe a Figura 8, nota-se que a maioria dos terremotos registrados que interferiram no Sirius tem uma distância de epicentro pequena, simbolizando regiões próximas. Isso faz sentido com a ideia de que quanto mais uma onda viaja mais ela dissipa sua energia. Além disso, olhando a cor dos terremotos com o crescimento da magnitude, nota-se que a frequência de pontos esverdeados e amarelos cresce na medida em que sobe-se na escala.

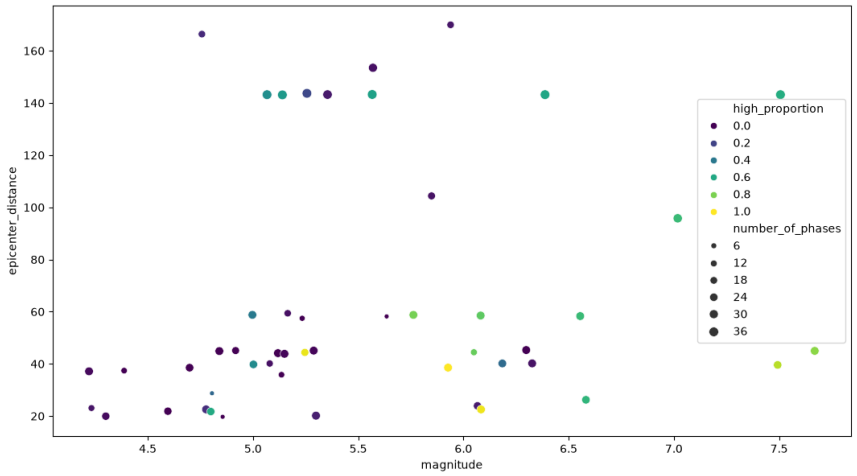


Figura 8: Gráfico de densidade dos terremotos, na abscissa temos a magnitude do evento e na ordenada a distância do epicentro em graus. Cada ponto tem duas informações adicionais: a cor do ponto representa a proporção de p-valores abaixo do limiar definido e o tamanho do ponto define o número de fases daquele terremoto.

Fonte: plot de autoria própria.

Ambas as afirmações eram expectativas antes de qualquer avaliação. Foi relatado pelos funcionários da sala de controle [9] que terremotos fora da América do Sul são raros de serem detectados e, ainda sim, os de baixa magnitude acabavam passando despercebido. Contudo, isso não significa que esse tipo de terremoto é especialmente influente nesse contexto, isso pode ser resultado de um viés de observação, já que só se fazia esse tipo de correlação quando o desvio do BPM se tornava grande o suficiente para o modulo de correção de orbita acusar erro.

Usando os dados disponíveis, os modelos foram treinados de forma preliminar, com hiper-parâmetros padrões para realizar uma avaliação inicial de cada um.

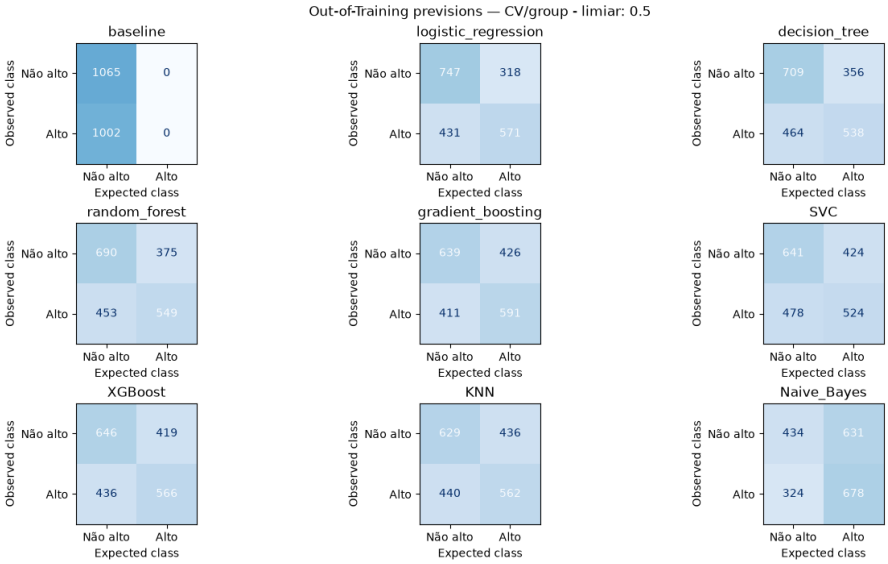


Figura 9: Resultados preliminares de cada um dos modelos treinados. Esse resultado leva em conda um limiar de 0.5 e hiper-parâmetros padrões.

Fonte: plot de autoria própria.

Antes de qualquer análise aprofundada, devemos pensar no que se deseja dessa análise. A princípio, o objetivo era usar um modelo treinado de classificação não só para correlacionar os eventos dos terremotos, mas para prever se um terremoto terá um impacto no Sirius e, por meio de uma segunda camada de modelos, prever a magnitude do desvio do feixe. Para isso, existem erros que podem significar mais que outros, receber um falso negativo simplesmente indica dados faltantes durante uma análise (no caso do modelo de previsão da magnitude do desvio), mas um falso positivo envia um dados errado para um treinamento posterior.

Por isso, será usada três métricas para avaliar o modelo: precisão, recall e Average Precision (AP). A precisão será avaliado como prioridade nesse teste, já que ela mede a confiabilidade de um dado positivo no teste. Contudo, se ela fosse uma métrica única, existiria a chance de um modelo ser selecionado de forma que ele chuta pouquíssimos positivo e estes coincidem com serem verdadeiros, para isso, será usado um Threshold do recall, que é a capacidade do modelo de recuperar os positivos verdadeiros. Assim, busca-se maximizar a precisão com a condição de que o Recall seja maior que 55%.

$$\text{precisão} = \frac{VP}{VP + FP} \quad ; \quad \text{recall} = \frac{VP}{VP + FN}$$

- VP: verdadeiros positivos
- FP: falsos positivos
- FN: falsos negativos

A AP é uma métrica secundaria, utilizada para saber se, em geral, um modelo melhorou depois de alguma mudança. Aqui vale diferenciar precisão de AP, a primeira depende do limiar definido para considerar uma categoria, a segunda faz uma média ponderada pelo incremento do recall de todos os limiares possíveis e obtém uma métrica global do modelo.

Com isso dito, a tabela 2 apresenta as estatísticas necessárias para uma análise preliminar. Em especial, três modelos se destacam nela: Regressão Logística, Gradient Boosting e o XGBoost. Os dois últimos são modelos que conseguem ultrapassar o limiar definido do recall sem ter uma

quantidade absurda de falsos positivos (característica que descarta o Naive Bayes como um bom modelo). Já a regressão logística tem uma precisão média muito elevada, isso unido de sua interpretabilidade o tronam um modelo interessante para definir como uma espécie de baseline linear.

Tabela 2: Desempenho dos modelos avaliados utilizando limiar de decisão igual a 0,50.

Modelo	AP	Precisão	Recall	FP	Folds na Meta
Baseline	0.485	0.000	0.000	0	0
Logistic Regression	0.702	0.642	0.514	306	3
Decision Tree	0.562	0.571	0.540	412	2
Random Forest	0.619	0.571	0.528	353	3
Gradient Boosting	0.612	0.551	0.569	438	3
SVC	0.556	0.481	0.446	385	2
XGBoost	0.638	0.579	0.550	378	3
KNN	0.587	0.538	0.536	455	3
Naive Bayes	0.569	0.529	0.665	626	3

Contudo, isso ainda é uma estimativa usando hiper-parâmetros não específicos. Para otimizar os hiper-parâmetros uma função auxiliar foi criada. Ela permite não só realizar a validação cruzada em uma grid usando o optuna, mas testar os limiares de decisão e filtrar candidatos sob uma hipótese do usuário. Para esse teste, foram feitas validações que mantivessem o recall médio acima de 55%, priorizassem a precisão e tivessem um número mínimo de FP ou uma boa quantidade de folds.

Para conferir os valores testados, veja a tabela 7. Os resultado dos modelos podem ser vistos nas tabelas 3 e 4. Em especial, destaca-se a regressão logística, o SVC e o Naive Bayes, que atingiram a faixa de 70% de precisão/recall ou tem uma AP alta. Isso é um resultado particularmente promissor, mas ainda não é preciso o suficiente para ser um modelo altamente confiável.

Tabela 3: Resultados da avaliação dos classificadores segundo o critério de precisão.

Modelo	Status	Limiar	Validation AP	Precision	Recall	Recall mínimo	FP	TP	Folds na meta
Baseline	elegível	0.20	0.485	0.485	1.000	1.000	1065	1002	5
Logistic Regression	elegível	0.45	0.702	0.701	0.680	0.341	292	685	4
Decision Tree	elegível	0.40	0.590	0.611	0.695	0.341	433	701	4
Random Forest	elegível	0.50	0.613	0.586	0.618	0.405	427	627	4
Gradient Boosting	elegível	0.50	0.647	0.646	0.698	0.341	382	704	4
SVC	elegível	0.50	0.682	0.715	0.737	0.627	323	738	5
XGBoost	elegível	0.40	0.642	0.594	0.641	0.411	431	650	4
KNN	elegível	0.55	0.574	0.563	0.581	0.300	442	588	4
Naive Bayes	elegível	0.35	0.724	0.665	0.663	0.432	358	671	4

Posteriormente, os hiper-parâmetros específicos encontrados para cada modelo serão expostos e discutidos. Agora, quais atributos realmente tem algum impacto na classificação dos modelos? É discutível se parâmetros como o ray_param realmente auxiliam na classificação ou só aumentam a dimensionalidade sem grandes ganhos de informação. Como forma de testar essa hipótese, será feita um teste de ablação e permutation importance.

Para reduzir o conjunto de dados analisado, só serão analisados os modelos ditos como promissores após a otimização de hiper-parâmetros. Veja a Figura 10, algo interessante de se notar é que os modelos se baseiam fortemente em dados relacionados a magnitude do evento,

Tabela 4: Resultados da avaliação dos classificadores segundo o critério de minimização de falsos positivos.

Modelo	Status	Limiar	Validation AP	Precision	Recall	Recall mínimo	FP	TP	Folds na meta
Baseline	elegível	0.20	0.485	0.485	1.000	1.000	1065	1002	5
Logistic Regression	elegível	0.45	0.698	0.701	0.675	0.341	291	680	4
Decision Tree	elegível	0.40	0.590	0.611	0.695	0.341	433	701	4
Random Forest	elegível	0.50	0.613	0.586	0.618	0.405	427	627	4
Gradient Boosting	elegível	0.50	0.646	0.646	0.698	0.341	382	704	4
SVC	elegível	0.55	0.680	0.704	0.647	0.351	274	652	4
XGBoost	elegível	0.40	0.634	0.591	0.623	0.405	430	631	4
KNN	elegível	0.50	0.587	0.524	0.569	0.056	425	583	4
Naive Bayes	elegível	0.35	0.724	0.665	0.663	0.432	358	671	4

em especial a representação pelo expected_amp. Tirando isso, parece que cada modelo tem uma preferência individual pelos atributos utilizados, vendo o resultado da regressão logística gera uma suspeita de que a otimização anterior tem uma forte relação com regularização l1, já que alguns pesos parecem ou ter sido zerados ou são insignificantes a análise. Naive Bayes parece conseguir retirar alguma informação útil, mesmo que o ganho na maioria dos casos seja risível de todos os atributos, com exceção de ray_param e Incident_angle. Por fim, SVC se beneficia de poucos atributos e se confunde com a maioria dos outros, usando os dados da magnitude e da fase para trassar as suas fronteiras.

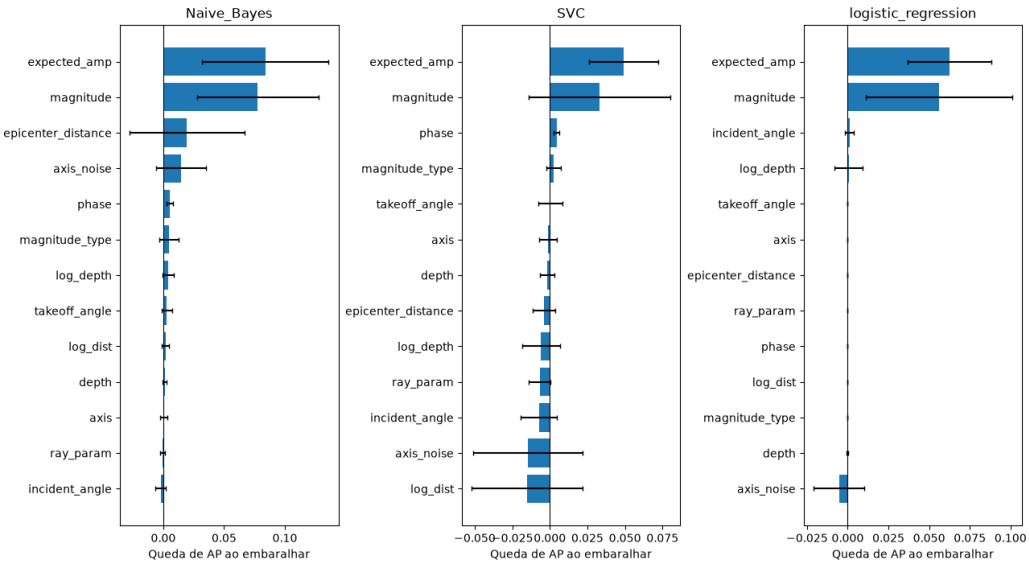


Figura 10: Resultado da variação da AP do modelo com um dado embaralhado. Note que a maioria das variáveis ajudam o modelo ou não impactam na medida, restando apenas algumas variáveis que um ou outro modelo tem pouco uso.

Fonte: plot de autoria própria.

Entretanto, todos os ganhos possíveis dessa análise não chegam a fornecer uma vantagem de nem 2% ao modelo. Logo, como forma de simplificação, o corte de alguns atributos seria válido, mas como uma estratégia de ganho de performance ainda é muito pequeno para ser justificável. Tal resultado é a motivação para testar um modelo reduzido de regressão logística e um Naive Bayes sem incident_angle, que apresenta uma melhora de quase 2% na precisão.

Ademais, vale ressaltar que um parâmetro aparecer com contribuição negativa dentro do teste não significa que ele realmente atrapalha o modelo. Existe a possibilidade de um fold ter tido um

desempenho muito inferior aos demais, resultando em uma média inferior quando, na realidade, ela seria mais próxima de uma neutralidade. O oposto também é válido, seria necessário testar para averiguar se as hipóteses são verdadeiras. Para uma avaliação mais detalhada dos valores, veja a tabela 8.

Retirando os atributos que tem contribuição negativa do modelos e aplicando os coeficiente achados pela validação cruzada, chegamos nos modelos representados na figura 11. O modelo de Naive Bayes parece ser claramente inferior aos outros dois por conta de seu recall, evidenciado pela tabela 5. Entretanto, lembre-se que o limiar optimal encontrado para ele está por volta do 35%. Logo, é esperado uma piora significativa, já que ele não está dentro de sua região ótima, ao contrario dos outros que estão perto dela.

Mesmo que um pouco promissor, esses não são os verdadeiros melhores modelos. Os resultados encontrados são específicos do espaço de busca e, se houve-se realmente a necessidade de encontrar o modelo ótimo para esse argumento, seria necessário incrementar a função de busca de hiper-parâmetros para testar combinações dos atributos que o modelo pudesse testar. Isso aumentaria o tempo de treino para uma questão de dias e, infelizmente, o computador de alto desempenho da ILUM, denominado de Heisenberg, estava indisponível durante o tempo de preparação desse projeto.

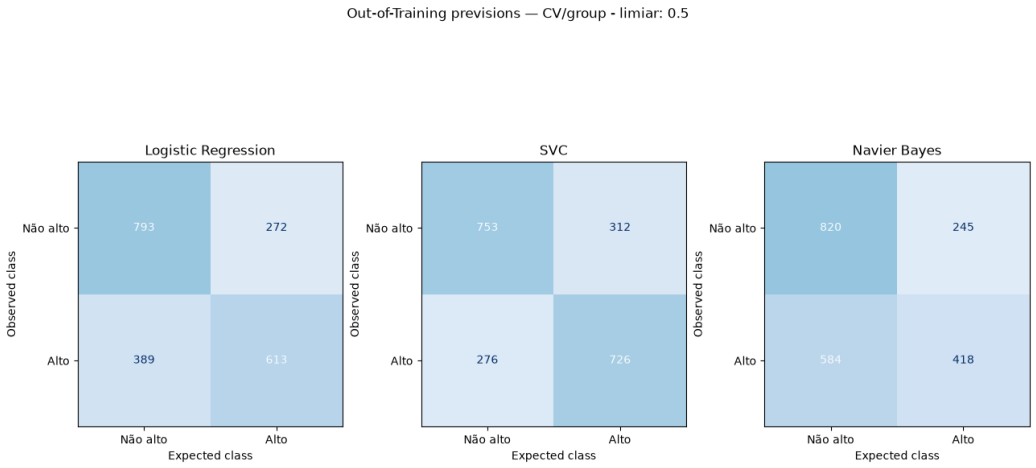


Figura 11: matriz de confusão dos modelos reduzidos. Note como alguns se tornaram melhores em achar positivos verdadeiros e outros apresentam um recall melhor do que o da figura 9. As matrizes mostradas não estão com o limiar optimal, por isso, ainda há espaço para melhora dentro deles.

Fonte: plot de autoria própria.

Tabela 5: Desempenho dos classificadores compactos.

Modelo	Limiar	AP	Precisão	Recall	FP	Folds na Meta
Logistic Regression	0.50	0.699	0.685	0.611	272	3
SVC	0.50	0.725	0.718	0.725	312	5
Naive Bayes	0.50	0.731	0.515	0.402	245	1

Com isso, as formas finais dos modelos de previsão de impacto de um terremoto.

A regressão logística foi configurada com C=0.005 e l1_ratio = 0.25. Ele recebe incident_angle, magnitude, log_depth e expected_amp como atributos e usa um limiar de 0.45.

O SVC recebe gamma=0.01 e C=0.001, utilizando os atributos phase, depth, axis, takeoff_angle, incident_angle, ray_param, magnitude, magnitude_type, epicenter_distance, log_depth e expected_amp e com um limiar de 0.55.

Por fim, Naive Bayer utiliza var_smoothing=1.0 e phase, depth, axis, takeoff_angle, magnitude, magnitude_type, epicenter_distance, log_depth, log_dist, axis_noise e expected_amp como atributos e um limiar de 0.35.

As informações relatadas anteriormente não entram em conflito com a tabela 5 ou com a Figura 11, já que retratam os modelos com o limiar fixo em 0.5, não significando que esse seja o limiar ideal dos modelos.

Treinando o modelo com os dados de treino e avaliando-o com a função predict obtêm-se o resultado exposto na tabela 6 e na Figura 12:

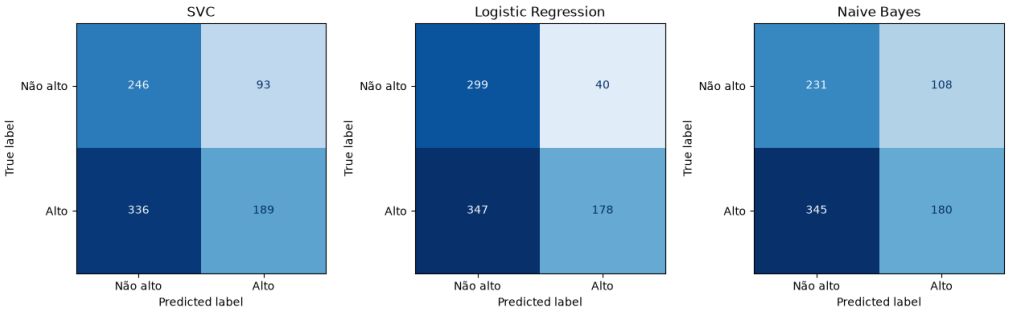


Figura 12: matriz de confusão dos modelos treinados e validados com o grupo de resposta. Nota-se uma visível queda no recall, mesmo que a precisão seja maior.

Fonte: plot de autoria própria.

Tabela 6: Desempenho dos modelos no conjunto de teste reservado.

Modelo	Limiar	AP	Precisão	Recall	FP	TP	FN	TN
SVC	0.50	0.707	0.670	0.360	93	189	336	246
Logistic Regression	0.45	0.738	0.817	0.339	40	178	347	299
Naive Bayes	0.35	0.702	0.625	0.343	108	180	345	231

A princípio, o resultado foge daquilo que a validação cruzada estava prometendo. Todos os modelos sofreram com uma perda enorme de recall para um limiar abaixo das regras impostas (recall $\geq$ 55%). Portanto, pelas regras estabelecidas, seria plausível afirmar que os modelos falharam nos requisitos mínimos para serem verdadeiramente preditivos. Entretanto, talvez exista um motivo para a validação com o grupo de teste ter sido tão distinta das estimativas anteriores.

Os dados são segmentados em grupos de cada evento, isso significa que, aproximadamente, 11 terremotos componham o grupo de teste. Isso fornece uma variabilidade muito baixa,

permitindo com que desvios estatísticos ocorram. Avaliando a composição desse grupo, nota-se que, aproximadamente, 60% das chegadas são classificadas como influentes, que difere dos 48% dos dados de treino. Possivelmente, isso é um indicador de que existe uma diferença entra ambos os grupos que pode fortalecer essa dificuldade em reconhecer verdadeiros positivos.

Analisando os pesos da regressão logística, a seguinte equação surge:

$$\eta \approx 0.3461 z_{magnitude} + 0.3261 z_{expected_amp} + 0.0793 z_{log_depth} - 0.0514$$

Posteriormente, o η é convertido em probabilidade pela função logística.

Nota-se que a regressão logística se baseia fortemente em argumentos potenciais do terremoto (baseados em magnitude) e pouco nas fases dos terremotos. Possivelmente o algoritmo tem pouca capacidade de distinguir fases que poderiam interferir com o sismógrafo e aposta em eventos suficientemente energéticos para causarem uma reação. Isso é fortalecido ao notar que nenhum argumento da função utiliza dados que discirnam entre componentes de um mesmo abalo. Utilizar poucas componentes também é coerente com os hiper-parâmetros selecionados, que tendem a uma forte regularização l1, o que, provavelmente, está zerando os outros atributos.

Infelizmente, não foi realizada uma análise adicional para interpretar os outros dois modelos no quesito de decisões individuais e em estrutura de decisão, mas, olhando novamente a Figura 10, somente o SVC e o Naive Bayes tem contribuição positiva de fases distintas, mesmo que ela seja baixa. Com isso, uma hipótese pode ser feito a respeito dos outros dois modelos: eles seguem um pensamento parecido ao da regressão linear. Talvez exista uma hiper-valorização pela energia do evento e menos visualização em informações de chegadas individuais. Talvez esse problema fosse solucionado buscando mais terremotos para compor o dataset ou usando modelos mais precisos para extrair mais informações de cada fase. Contudo, isso não pode ser claramente visualizado nesse trabalho.

Vale ressaltar que tudo isso faz parte de uma especulação do autor. Para confirmar qualquer um dos pontos mencionados mais testes são necessários e uma apuração mais rigorosa dos métodos de tratamento deveria ser feita. Todavia, o projeto não buscou se aprofundar nesse quesito.

Conclusões e principais aprendizados

Sobre o objetivo de encontrar uma correlação entre terremotos e resposta no sismógrafo do Sirius, o projeto encontrou candidatos possíveis para tal avaliação, mas obteve soluções abaixo das metas estabelecidas e não atingiram a sensibilidade mínima estabelecida.

Os dados analisados aparentam demonstrar algum tipo de tendência quanto a classificação de scores e dados da onda, vide Figura 7 e 8. Essa suspeita parecia se fortificar com os resultados preliminares dos modelos reduzidos expostos na tabela 5, mas não pode ser devidamente validada por problemas durante a etapa de validação com o grupo de testes selados, analisado na tabela 6.

O trabalho conclui que existe um potencial dentro da abordagem, mas será necessário validar o sistema de correlação de ondas, avaliando se não é uma métrica enviesada de geração de p-valores baixos, expandir o valor informativo do modelo de ondas de corpo, para fornecer geometria do evento e as frequências de chegada, e, mais importante, correlacionar o sismógrafo com desvios anômalos dos BPMs. Em especial, esse último poderia ajudar a filtrar picos que implicam em alguma mudança e picos que não tem impacto direto dentro das linhas de luz. Infelizmente, o projeto não conseguiu avançar o suficiente para utilizar os BPMs, sendo esse um futuro objetivo do projeto.

Além disso, seria necessário estudar melhor a queda entre a estimativa e a validação efetiva, já que a discussão permanece como hipótese.

Para finalizar, o aluno conseguiu aprofundar muito o seu conhecimento não só em teorias novas, como a física de placas tectônicas e a propagação de ondas sísmicas, mas dentro da própria matéria de Machine Learning, ganhando prática com ferramentas que nunca tinha utilizado, analisando parâmetros e exercitando o raciocínio crítico ao interpretar modelos matemáticos e dados e testando modelos que nunca havia estudados sobre.

Apêndice

Tabela 7: Espaço de busca utilizado para cada classificador.

Modelo	**Hiperparâmetros avaliados**
Logistic Regression	$C \in [0.003, 1.0]$ (20 valores); $l1_ratio \in \{0, 0.25, 0.5, 0.75, 1\}$
Decision Tree	$max_depth \in \{2, 3, 4, 6, 8\}$; $min_samples_leaf \in \{5, 10, 20, 30, 50\}$; $ccp_alpha \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$
Random Forest	$max_depth \in \{3, 5, 8, 12, \text{None}\}$; $min_samples_leaf \in \{5, 10, 20, 30, 50\}$; $max_features \in \{\text{sqrt}, 0.5, 0.75, 1.0\}$
HistGradientBoosting	$learning_rate \in \{0.01, 0.03, 0.05, 0.1, 0.3\}$; $max_iter \in \{50, 100, 200, 400, 600\}$; $max_depth \in \{1, 2\}$; $l2_regularization \in \{1, 10\}$
SVC	$C \in \{10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}, 3\times10^{-1}, 1, 3, 10, 100\}$; $\gamma \in \{\text{scale}, 10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}, 3\times10^{-1}, 1\}$
XGBoost	$n_estimators \in \{100, 200, 400, 600, 800\}$; $max_depth \in \{2, 3, 4, 6, 8\}$; $learning_rate \in \{0.03, 0.1\}$; $subsample \in \{0.7, 1.0\}$
KNN	$n_neighbors \in \{3, 5, 7, 11, 15, 21, 31, 51, 75, 101\}$; $weights \in \{\text{uniform}, \text{distance}\}$; $metric \in \{\text{euclidean}, \text{manhattan}, \text{chebyshev}, \text{canberra}, \text{braycurtis}\}$
Gaussian Naive Bayes	$var_smoothing \in [10^{-12}, 1]$; 100 valores em escala logarítmica

Tabela 8: Importância por permutação dos atributos, expressa como queda média de AP ± desvio-padrão entre os cinco folds de validação.

Atributo	Naive Bayes	SVC	Regressão logística
`expected_amp`	0.084 ± 0.052	0.049 ± 0.023	0.063 ± 0.026
`magnitude`	0.078 ± 0.050	0.033 ± 0.047	0.056 ± 0.045
`epicenter_distance`	0.020 ± 0.047	−0.004 ± 0.008	0.000 ± 0.000
`axis_noise`	0.015 ± 0.020	−0.015 ± 0.036	−0.005 ± 0.016
`phase`	0.006 ± 0.003	0.004 ± 0.002	0.000 ± 0.000
`magnitude_type`	0.005 ± 0.008	0.003 ± 0.005	0.000 ± 0.000
`log_depth`	0.004 ± 0.005	−0.006 ± 0.013	0.001 ± 0.008
`takeoff_angle`	0.003 ± 0.004	0.000 ± 0.008	0.000 ± 0.000
`log_dist`	0.002 ± 0.003	−0.015 ± 0.037	0.000 ± 0.000
`depth`	0.001 ± 0.002	−0.002 ± 0.005	−0.000 ± 0.000
`axis`	0.000 ± 0.003	−0.001 ± 0.006	0.000 ± 0.000
`ray_param`	−0.001 ± 0.002	−0.007 ± 0.007	0.000 ± 0.000
`incident_angle`	−0.002 ± 0.004	−0.007 ± 0.012	0.001 ± 0.003

Descrição do uso de IA neste trabalho

Durante o desenvolvimento desse projeto, três modelos de linguagem foram utilizados em quatro conversas distintas, além do modelo do Gemini para ler e interpretar os dados do Olog.

Copilot: essa IA serviu como um verificador rápido de erros e um ajudante na escritura do código, com o objetivo de instruir como algumas funções mais obscuras funcionam e que argumentos elas recebem. Nenhuma linha de código foi explicitamente escrita com ele, servindo como uma auxiliar durante a programação.

ChatGPT: dois chats distintos foram usados nesse modelo, um visava dar o embasamento teórico e ajudou a buscar por literatura e bibliotecas relevantes, sendo a sugestão de usar o `ObsPy` advinda dele. A conversa está disponível em: ChatGPT/teórico.

O segundo chat foi usado lado a lado do estudante para formular e desenvolver todo o programa. Ele auxiliou na analise de tabelas de arquivos, buscando erros e inconsistências na formulação. O relatório final da atividade também foi submetido a ele como forma de análise crítica do conteúdo. Além disso, alguns scripts foram formulados por ele usando o código do aluno de base, como o `model_searches` e `feature_analysis`, que consistem em compilados de testes realizados dentro de `seismo_detection` (visando a organização e eficiência), e o `optuna_searches` e `final_model`, que foram escritos para poupar tempo na hora de repetir algumas análises já realizadas. Entretanto, declaro que o aluno tem plena consciência e responsabilidade perante ao conteúdo produzido pela inteligência artificial, exercendo o seu senso crítico durante os usos do modelo. A conversa encontra-se disponível em: ChatGPT/acompanhante.

Claude: o Claude serviu como um segundo juiz ao programa sendo escrito, já que ele fornece perspectivas e um rigor distinto do ChatGPT. Ele que sugeriu a forma de tratamento da onda por envelope e que incentivou o estudante a mudar o modelo original de calcular score para um que utiliza amostragem aleatória de intervalos não pertencentes ao original. A conversa com ele está disponível em: Claude.ia.

Referências

1 CHARACTERIZATION of tidal and seismic effects on the beam energy and orbit of the High Energy Photon Source. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, dez. 2026. Accessed: 2026-10-25. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0168900226005322>.

2 LUMPKIN, A. H.; YANG, B. X.; YAO, C. Y. X-ray imaging of the aps storage ring beam stability effects: From the alaskan earthquake to undulator field changes. In: *Proceedings of the 2003 Particle Accelerator Conference (PAC 2003)*. [S.l.: s.n.], 2003. p. 2423–2425.

3 SHEARER, P. M. *Introduction to Seismology*. 2. ed. Cambridge: Cambridge University Press, 2009. ISBN 9780521882101.

4 AKI, K.; RICHARDS, P. G. *Quantitative Seismology*. 2. ed. Mill Valley, California: University Science Books, 2009. ISBN 9781891389634.

5 U.S. Geological Survey. *Magnitude Types*. 2020. <https://www.usgs.gov/programs/earthquake-hazards/magnitude-types>. Accessed: 2026-10-24.

6 Physical Society of Thailand. *Seismic Waves: The Key to Understanding the Earth's Interior, Part 1*. 2019. Acesso em: 20 out. 2026. Disponível em: <https://www.thaiphysoc.org/article/73/>.

7 ObsPy Development Team. *obspy.clients.fdsn – FDSN Web Services Client for ObsPy*. 2023. <https://docs.obspy.org/archive/1.4.1/packages/obspy.clients.fdsn.html>. ObsPy 1.4.1 Documentation. Acesso em: 20 out. 2026.

8 EPICS Archiver Appliance. *User Guide: Retrieving Data Using Other Tools*. n.d. <https://epicsarchiver.readthedocs.io/en/stable/user/userguide.html#retrieving-data-using-other-tools>. Accessed: 2026-10-10.

9 BENTO, W. L. G.; MUKAI, A. H. C. *Informação sobre o período de amostragem dos sismógrafos e BPMs*. 2026. Comunicação pessoal realizada por mensagem eletrônica em outubro de 2026.

10 EKSTRöM, G. A global model of love and rayleigh surface wave dispersion and anisotropy, 25–250 s. *Geophysical Journal International*, v. 187, n. 3, p. 1668–1686, 2011.

11 KENNETT, B. L. N.; ENGDAHL, E. R. Traveltimes for global earthquake location and phase identification. *Geophysical Journal International*, v. 105, n. 2, p. 429–465, 1991.

12 WANG, Y. et al. Deep learning for p-wave arrival picking in earthquake early warning. *Earthquake Engineering and Engineering Vibration*, v. 20, n. 2, p. 391–402, 2021.

13 VIRTANEN, P. et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, v. 17, n. 3, p. 261–272, 2020.

14 CASSAR, D. R. *biblioteca_real*. 2026. GitHub repository. Repositório dos materiais didáticos preparados pelos estudantes da Ilum Escola de Ciência. Acesso em: 25 out. 2026. Disponível em: <https://github.com/drcassar/biblioteca_real>.

15 CUTOLO, V. M. d. M. *Regularização*. 2026. <https://github.com/drcassar/biblioteca_real/tree/main/Aprendizado%20de%20M%C3%A1quina/Regulariza%C3%A7%C3%A3o/T26%20-%20Vin%C3%ADcius%20Marianno%20De%20Marque%20Cutolo>. Material didático da Ilum Escola de Ciência, disponível no repositório Biblioteca Real. Acesso em: 25 out. 2026.

16 Sá, P. L. M. d. S. d. *Gradient Boosting*. 2026. Jupyter Notebook. Material didático da Ilum Escola de Ciência. Acesso em: 25 out. 2026. Disponível em: <https://github.com/drcassar/biblioteca_real/blob/main/Aprendizado%20de%20M%C3%A1quina/Gradient%20Boosting/T26%20-%20Pedro%20Lucas%20Miranda%20da%20Silva%20de%20S%C3%A1.ipynb>.

17 SOUZA, M. L. M. d. *XGBoost*. 2026. <https://github.com/drcassar/biblioteca_real/tree/main/Aprendizado%20de%20M%C3%A1quina/XGBoost/T26%20-%20Matheus%20Luiz%20Mendes%20de%20Souza>. Material didático da Ilum Escola de Ciência, disponível no repositório Biblioteca Real. Acesso em: 25 out. 2026.

18 NAVARRO, L. *Naive Bayes*. 2026. Jupyter Notebook. Material didático da Ilum Escola de Ciência, disponível no repositório biblioteca_real. Acesso em: 25 out. 2026. Disponível em: <https://github.com/drcassar/biblioteca_real/blob/main/Aprendizado%20de%20M%C3%A1quina/Naive%20Bayes/T26%20-%20Luis%20Navarro.ipynb>.